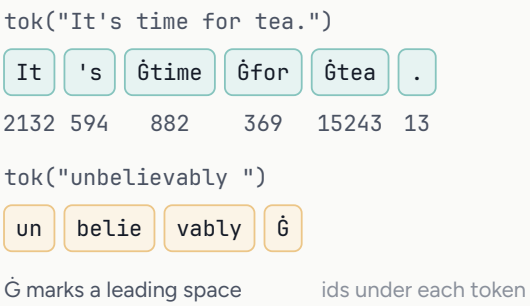
SECTION 1, PART 2 CHEAT SHEET

A decoder scores every possible next token

Text becomes token ids and ids become vectors. Attention mixes the tokens, the MLP works on each one, and the last position scores all 151,936 next tokens. Generation repeats that in a loop; training lowers the loss of the real next token.



1 TOKENS

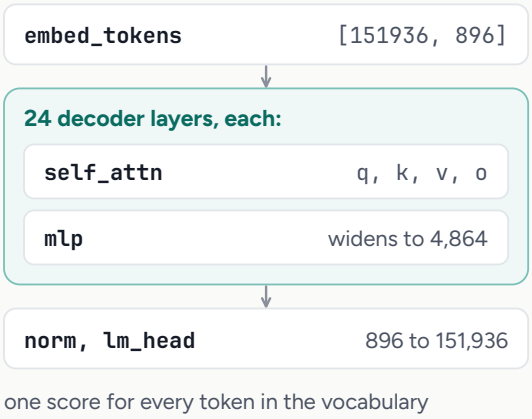
Text becomes token ids

The tokenizer cuts text into tokens it knows and returns one id per token. A rare word splits into pieces; `Ġ` marks a leading space, so `Ġtea` and `tea` are different tokens.

```
MODEL = "Qwen/Qwen2.5-0.5B-Instruct"
tok = AutoTokenizer.from_pretrained(MODEL)
model = AutoModelForCausalLM.from_pretrained(MODEL,
    device_map="cuda")

ids = tok("It's time for tea.")["input_ids"]
tok.convert_ids_to_tokens(ids)
len(tok)
```

COURSE RUN the tokenizer knows **151,665** tokens.



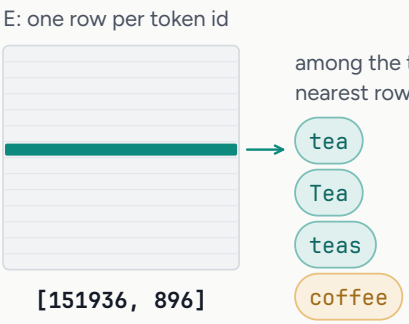
2 THE MODEL'S PARTS

From ids to scores

Printing the model shows its parts: an embedding table, 24 identical decoder layers of attention and an MLP, then `lm_head`, which turns each position's numbers into one score per token.

```
model
layer = model.model.layers[0]
layer
```

COURSE RUN `embed_tokens` is **[151936, 896]**; the MLP widens 896 numbers to **4,864** and back.



3 EMBEDDINGS

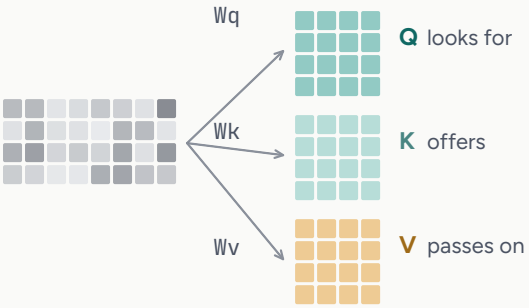
Each id picks one row of numbers

The embedding matrix has one row per token id. Rows for tokens used alike sit close together, which cosine similarity measures.

```
E = model.model.embed_tokens.weight.float()
tea_id = tok.encode(" tea")[0]
tea = E[tea_id]
similarity = torch.cosine_similarity(E, tea[None], dim=-1)
top = similarity.topk(10)
```

COURSE RUN one token is **896** numbers; **coffee** is among the ten nearest rows to `Ġtea`.

X: 4 tokens, 8 numbers each



Q, K and V are each [4, 4]; each row of the softmax weights adds up to 1

4 QUERY, KEY, VALUE

Three matrices, three roles

Three learned matrices turn each token's numbers into a query (what it looks for), a key (what it offers) and a value (what it passes on). Attention compares queries with keys and mixes the values.

```
torch.manual_seed(0)
X = torch.randn(4, 8)
Wq = torch.randn(8, 4) / math.sqrt(8)
Q = X @ Wq
K = X @ Wk
V = X @ Wv
weights.sum(dim=-1)
```

COURSE RUN `Q.shape` is **[4, 4]**; `weights.sum(dim=-1)` is **1** for every row.

causal_weights: who each token looks at

1	**1.00**	0	0	0
2	0.25	**0.75**	0	0
3	0.31	0.26	0.42	0
4	0.08	0.29	0.09	**0.54**

above the diagonal: masked to 0
every row adds up to 1

5 ATTENTION BY HAND

Scores, scaling, softmax, the causal mask

softmax(QK^T / √d) V: every token scores every other, softmax turns each row into weights, and the output mixes the values. In a decoder the mask hides the tokens that come later.

```
scores = (Q @ K.T) / math.sqrt(4)
weights = torch.softmax(scores, dim=-1)
out = weights @ V

mask = torch.tril(torch.ones(4, 4)).bool()
masked = scores.masked_fill(~mask, float("-inf"))
causal_weights = torch.softmax(masked, dim=-1)
torch.nn.functional.scaled_dot_product_attention(
    Q, K, V, is_causal=True)
```

COURSE RUN token 2 looked at token 4 with **0.4436** before the mask; after it, only tokens 1 and 2 (**0.2520**, **0.7480**). PyTorch's built-in gives the same rows as `causal_weights @ V`.

one token: 896 numbers, 14 heads of 64



one transformer block, weights



6 HEADS AND THE BLOCK

Many heads, then the MLP

Each token's numbers split into heads that attend in parallel. A transformer block is attention plus the MLP; Qwen stacks 24.

```
heads = x.view(5, 14, 64)
model.set_attn_implementation("eager")
response = model(**x, output_attentions=True)
len(response.attentions), response.attentions[0].shape
```

COURSE RUN **24** maps of **[1, 14, 11, 11]**; the MLP holds about 7 times the weights.



of all 151,936 next tokens, the top three

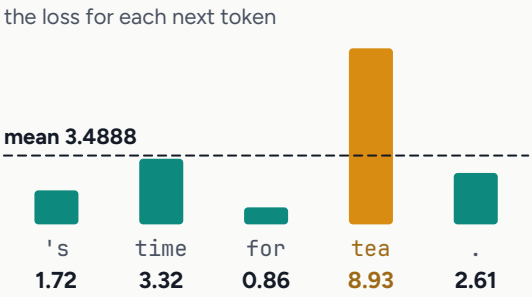
7 LOGITS AND GENERATION

Score the next token, then loop

The last position scores every token in the vocabulary, and earlier words change the scores. Generation appends the top token and runs again.

```
logits = model(**x).logits
last = logits[0, -1].float()
probs = last.softmax(dim=-1)
top = probs.topk(5)
next_id = logits[0, -1].argmax()
ids = torch.cat([ids, next_id.view(1, 1)], dim=1)
response = model.generate(**x, max_new_tokens=8,
    do_sample=False, repetition_penalty=1.0)
```

COURSE RUN loop and `generate` agree: “a cup of tea. I'll go to the kitchen”.



8 TRAINING

The next-token loss

Every position predicts the next token. The loss is **−(1/N) Σ log p** of the real next tokens: cross-entropy, averaged.

```
response = model(**x, labels=x["input_ids"])
response.loss
predictions = response.logits[0, :-1].float()
targets = x["input_ids"][0, 1:]
torch.nn.functional.cross_entropy(predictions, targets,
                                  reduction="none")
```

COURSE RUN loss **3.4888**, the mean of five; `for` to `tea` costs **8.9251**.

TERMS

token id: the number that stands for a token.

embedding: the numbers one token becomes.

query, key, value: what a token looks for, offers and passes on.

attention head: one of the parallel attentions.

causal mask: hides later words from a decoder.

logits: raw scores, before softmax.

WHAT EACH CALL HANDS BACK

`embed_tokens.weight`
[151936, 896] ids by numbers

`model(**x).logits`
[1, 4, 151936] text, tokens, vocabulary

`model(**x, labels=...).loss`
one number, the mean

WATCH OUT

- The default attention cannot hand back its weights: switch to `"eager"` first.
- Qwen's saved settings sample and penalise repeats: `do_sample=False` and `repetition_penalty=1.0` make `generate` match the loop.